

Code Source Complet

Agents 38 a 42 - logique métier et interface Streamlit de chaque projet.

38. SiteMap-Firecrawl

38-SiteMap-Firecrawl/map_agent.py

```
import os
from textwrap import dedent

from dotenv import load_dotenv
from firecrawl import Firecrawl
from pydantic import BaseModel, Field

from agno.agent import Agent
from agno.models.groq import Groq

load_dotenv()

FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

# map() ne scrape aucun contenu (juste la découverte des URLs) : c'est l'appel
# Firecrawl le moins coûteux, on peut se permettre une limite plus large que
# pour search()/crawl() sans exploser le budget de crédits.
MAP_LIMIT = 40

firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)

class Link(BaseModel):
    url: str
    title: str

class Category(BaseModel):
    name: str = Field(description="Nom de la catégorie (ex: Documentation, Blog, Produit, API...)")
    urls: list[str] = Field(description="URLs de cette catégorie, reprises exactement telles que fournies")

class SiteMapResult(BaseModel):
    categories: list[Category]

categorizer_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Site Map Categorizer",
    role="Expert en architecture de site web",
    instructions=dedent("""
    Vous recevez une liste d'URLs (avec leur titre) découvertes sur un site web via
    l'outil de cartographie Firecrawl (map). Votre mission : les regrouper en
    catégories logiques et lisibles (ex: Documentation, Blog, Produit, API Reference,
    Tarifs, À propos, Légal, Autre...).

    Règles strictes :
    - Ne reformulez JAMAIS une URL : reprenez-la exactement telle qu'elle apparaît
      dans la liste fournie.
    - N'inventez aucune URL qui ne serait pas dans la liste d'entrée.
    - Choisissez des noms de catégories clairs, adaptés au site observé (ne pas
      forcer des catégories génériques si elles ne correspondent à rien).
    - Une URL qui ne correspond à aucune catégorie claire va dans "Autre".
    - Triez les catégories par nombre d'URLs décroissant.
    """),
    output_schema=SiteMapResult,
)

def explore_site(url: str, search: str | None = None, limit: int = MAP_LIMIT) -> list[Link]:
    """Découvre les URLs d'un site via Firecrawl map (rapide, sans scraping de contenu)."""
    result = firecrawl.map(url, search=search, limit=limit)
    links = []
    for item in result.links or []:
        if not item.url:
            continue
        links.append(Link(url=item.url, title=item.title or item.url))
```

```

return links

def categorize(links: list[Link] -> list[Category]:
    """Demande au LLM de regrouper les liens découverts en catégories lisibles."""
    if not links:
        return []

    links_block = "\n".join(f"{l.url} | {l.title}" for l in links)
    prompt = f"URLs découvertes ({len(links)}) :\n\n{links_block}"

    result = categorizer_agent.run(prompt)
    if not isinstance(result.content, SiteMapResult):
        # Le modèle n'a pas renvoyé de JSON structuré valide : on retombe sur une
        # unique catégorie "Toutes les pages" plutôt que de faire planter l'UI.
        return [Category(name="Toutes les pages", urls=[l.url for l in links])]
    return result.content.categories

def map_site(url: str, search: str | None = None, limit: int = MAP_LIMIT) -> tuple[list[Link],
list[Category]]:
    """Découvre puis catégorise les URLs d'un site. Retourne (liens bruts, catégories)."""
    links = explore_site(url, search=search, limit=limit)
    categories = categorize(links)
    return links, categories

```

38-SiteMap-Firecrawl/app.py

```

import streamlit as st
from map_agent import map_site, MAP_LIMIT

st.set_page_config(page_title="Cartographie de Site", page_icon="♦", layout="wide")

st.markdown(
    """
    <style>
    @import
        url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

    :root {
        --ink: #0f172a; --muted: #64748b; --accent: #1d4ed8; --accent-solid: #1d4ed8;
        --border: #e2e8f0; --surface: rgba(15, 23, 42, 0.02); --card-bg: #ffffff;
    }
    @media (prefers-color-scheme: dark) {
        :root {
            --ink: #f1f5f9; --muted: #94a3b8; --accent: #60a5fa;
            --border: rgba(255, 255, 255, 0.14); --surface: rgba(255, 255, 255, 0.04);
            --card-bg: #1e293b;
        }
    }

    html, body, [class*="css"] { font-family: 'Inter', -apple-system, sans-serif; }
    h1, h2, h3 { font-family: 'Inter', sans-serif !important; font-weight: 700 !important; }
    .block-container { padding-top: 2.2rem; max-width: 880px; }

    .eyebrow { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.12em; text-transform:
        uppercase; color: var(--accent); margin-bottom: 0.5rem; }
    .hero-title { font-weight: 700; font-size: 2rem; color: var(--ink); margin-bottom: 0.35rem;
        line-height: 1.25; }
    .hero-subtitle { color: var(--muted); font-size: 0.98rem; margin-bottom: 1.4rem; max-width:
        640px; line-height: 1.5; }
    .hero-rule { border: none; border-top: 1px solid var(--border); margin: 0 0 1.8rem 0; }

    div[data-testid="stTextInput"] input {
        border-radius: 6px !important; padding: 0.7rem 0.9rem !important; font-size: 0.98rem
        !important;
        border: 1px solid var(--border) !important; box-shadow: none !important; transition:
        border-color .12s ease;
    }
    div[data-testid="stTextInput"] input:focus {
        border-color: var(--accent) !important; box-shadow: 0 0 3px rgba(29, 78, 216, 0.12)
        !important;
    }

    div[data-testid="stFormSubmitButton"] button {
        background: var(--accent-solid) !important; color: white !important; border: none
        !important;
        border-radius: 6px !important; padding: 0.6rem 1.4rem !important; font-weight: 600
        !important;
        letter-spacing: 0.01em; transition: background .12s ease;
    }
    div[data-testid="stFormSubmitButton"] button:hover { background: #1e40af !important; }

```

```


[data-testid="stFormSubmitButton"] button p { color: white !important; font-weight: 600; }

div[class*="st-key-cat_"] {
    border-radius: 8px !important; border: 1px solid var(--border) !important;
    background: var(--surface) !important; box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
}

.cat-title { font-size: 0.78rem; font-weight: 600; text-transform: uppercase;
    letter-spacing: 0.08em; color: var(--accent); margin-bottom: 0.6rem; }
.cat-count { color: var(--muted); font-weight: 400; text-transform: none; letter-spacing: 0; }

.link-row { display: block; padding: 0.35rem 0; border-bottom: 1px solid var(--border);
    font-size: 0.86rem; }
.link-row:last-child { border-bottom: none; }
.link-row a { color: var(--ink); text-decoration: none; }
.link-row a:hover { color: var(--accent); }

.summary-bar {
    display: flex; gap: 1.5rem; padding: 0.9rem 1.1rem; border: 1px solid var(--border);
    border-radius: 8px; background: var(--surface); margin-bottom: 1.4rem; font-size:
        0.85rem; color: var(--muted);
}
.summary-bar b { color: var(--ink); }

.empty-state { padding: 2.4rem 1rem; text-align: center; color: var(--muted); border: 1px
    dashed var(--border); border-radius: 8px; font-size: 0.92rem; }
</div></div>
"""
unsafe_allow_html=True,
)

st.markdown('<div class="eyebrow">Firecrawl · map()</div>', unsafe_allow_html=True)
st.markdown('<div class="hero-title">Cartographie de Site</div>', unsafe_allow_html=True)
st.markdown(
    '<div class="hero-subtitle">Découvrez toutes les URLs d'un site (sans scraper leur contenu) '
    'et les regroupe automatiquement en catégories lisibles.</div>',
    unsafe_allow_html=True,
)

st.markdown('<hr class="hero-rule" />', unsafe_allow_html=True)

with st.sidebar:
    st.markdown("### Paramètres")
    limit = st.slider("Nombre d'URLs max", min_value=10, max_value=100, value=MAP_LIMIT,
        step=10)
    search_filter = st.text_input("Filtrer par mot-clé (optionnel)", placeholder="Ex : pricing")
    st.caption("<map() est l'appel Firecrawl le moins coûteux : il ne lit pas le contenu des
        pages.>")

with st.form("map_form"):
    url = st.text_input(
        "URL du site :",
        placeholder="Ex : https://docs.firecrawl.dev",
        label_visibility="collapsed",
    )
    submitted = st.form_submit_button("Cartographier le site")

if "result" not in st.session_state:
    st.session_state.result = None

if submitted and url.strip():
    with st.spinner("Découverte des URLs en cours..."):
        links, categories = map_site(url.strip(), search=search_filter.strip() or None,
            limit=limit)
        st.session_state.result = {"url": url.strip(), "links": links, "categories": categories}

if not st.session_state.result:
    st.markdown(
        '<div class="empty-state">Aucune cartographie pour l'instant. '
        'Saisissez une URL de site ci-dessus pour commencer.</div>',
        unsafe_allow_html=True,
    )
else:
    res = st.session_state.result
    st.markdown(
        f'<div class="summary-bar">'
        f'<span>&#x2014; <b>{len(res["links"])}</b> URLs découvertes</span>'
        f'<span>&#x2014; <b>{len(res["categories"])}</b> catégories</span>'
        f'<span>Site : <b>{res["url"]}</b></span>'
        "</div>",
        unsafe_allow_html=True,
    )

if not res["categories"]:


```

```

st.markdown(
    '<div class="empty-state">Aucune URL trouvée pour ce site.</div>',
    unsafe_allow_html=True,
)
else:
    cols = st.columns(2)
    for i, cat in enumerate(res["categories"]):
        with cols[i % 2]:
            with st.container(border=True, key=f"cat_{i}"):
                st.markdown(
                    f'<div class="cat-title">{cat.name} '
                    f'<span class="cat-count">({len(cat.urls)})</span></div>',
                    unsafe_allow_html=True,
                )
            rows = "".join(
                f'<div class="link-row"><a href="{u}" target="_blank">{u}</a></div>'
                for u in cat.urls[:15]
            )
            if len(cat.urls) > 15:
                rows += f'<div class="link-row">... et {len(cat.urls) - 15} de plus</div>'
            st.markdown(rows, unsafe_allow_html=True)

```

39. SiteCrawl-Firecrawl

39-SiteCrawl-Firecrawl/crawl_agent.py

```
import os
from textwrap import dedent
from typing import Optional

from dotenv import load_dotenv
from firecrawl import Firecrawl
from firecrawl.v2.types import ScrapeOptions
from pydantic import BaseModel

from agno.agent import Agent
from agno.models.groq import Groq

load_dotenv()

FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

# crawl() scrape RÉELLEMENT chaque page visitée (contrairement à map) : c'est
# l'appel le plus coûteux en crédits Firecrawl et en tokens Groq, donc la
# limite par défaut reste volontairement basse.
CRAWL_LIMIT = 5
MAX_CHARS_PER_PAGE = 2500

firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)

class Page(BaseModel):
    title: str
    url: str
    content: str

digest_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Site Digest Writer",
    role="Expert en analyse et synthèse de sites web",
    instructions=dedent("""
    Vous recevez plusieurs pages numérotées (titre, URL, contenu) toutes issues du
    MÊME site web, obtenues par exploration automatique (crawl). Votre mission :
    produire une fiche de synthèse du site, basée UNIQUEMENT sur ces pages.

    Contenu attendu, en Markdown :

    ## Vue d'ensemble
    <de quoi parle ce site / cette section, en 2-3 phrases>

    ## Thèmes principaux
    <liste des sujets/sections identifiés, avec citation [n] de la ou des pages
    qui en parlent>

    ## Pages notables
    <2 à 5 pages qui semblent les plus importantes, avec une phrase sur chacune>

    Règles strictes :
    - N'inventez rien qui ne soit pas dans les pages fournies.
    - Citez systématiquement vos affirmations avec [1], [2]... renvoyant aux pages.
    - Si le contenu fourni est trop pauvre pour une vraie synthèse, dites-le
      clairement plutôt que de gonfler artificiellement la réponse.
    """),
    markdown=True,
)

def crawl_site(url: str, limit: int = CRAWL_LIMIT) -> list[Page]:
    """Explore un site à partir de `url` (en suivant les liens) et scrape chaque page
    visitée."""
    job = firecrawl.crawl(url, limit=limit, scrape_options=ScrapeOptions(formats=["markdown"]))

    pages = []
    for doc in job.data or []:
        markdown = (getattr(doc, "markdown", None) or "").strip()
        metadata = getattr(doc, "metadata", None)
        page_url = getattr(metadata, "url", None) if metadata else None
        title = (getattr(metadata, "title", None) if metadata else None) or page_url

        if not page_url or not markdown:
            continue
```

```

        pages.append(Page(title=title, url=page_url, content=markdown[:MAX_CHARS_PER_PAGE]))
    return pages

def digest(url: str, limit: int = CRAWL_LIMIT) -> tuple[str, list[Page]]:
    """Crawl le site puis demande au LLM une fiche de synthèse sourcée. Retourne (fiche,
    pages)."""
    pages = crawl_site(url, limit=limit)
    if not pages:
        return "Aucune page exploitable n'a été trouvée lors de l'exploration de ce site.", []

    pages_block = "\n\n".join(
        f"[{i}] {p.title} ({p.url})\n{p.content}" for i, p in enumerate(pages, 1)
    )
    prompt = f"Site exploré : {url}\n\nPages :\n\n{pages_block}"

    report = digest_agent.run(prompt)
    return report.content, pages

```

39-SiteCrawl-Firecrawl/app.py

```

import streamlit as st
from crawl_agent import digest, CRAWL_LIMIT

st.set_page_config(page_title="Exploration de Site", page_icon="♦", layout="wide")

st.markdown(
    """
    <style>
    @import
        url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

    :root {
        --ink: #0f172a; --muted: #64748b; --accent: #1d4ed8; --accent-solid: #1d4ed8;
        --border: #e2e8f0; --surface: rgba(15, 23, 42, 0.02); --card-bg: #ffffff;
    }
    @media (prefers-color-scheme: dark) {
        :root {
            --ink: #f1f5f9; --muted: #94a3b8; --accent: #60a5fa;
            --border: rgba(255, 255, 255, 0.14); --surface: rgba(255, 255, 255, 0.04);
            --card-bg: #1e293b;
        }
    }

    html, body, [class*="css"] { font-family: 'Inter', -apple-system, sans-serif; }
    h1, h2, h3 { font-family: 'Inter', sans-serif !important; font-weight: 700 !important; }
    .block-container { padding-top: 2.2rem; max-width: 880px; }

    .eyebrow { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.12em; text-transform:
        uppercase; color: var(--accent); margin-bottom: 0.5rem; }
    .hero-title { font-weight: 700; font-size: 2rem; color: var(--ink); margin-bottom: 0.35rem;
        line-height: 1.25; }
    .hero-subtitle { color: var(--muted); font-size: 0.98rem; margin-bottom: 1.4rem; max-width:
        640px; line-height: 1.5; }
    .hero-rule { border: none; border-top: 1px solid var(--border); margin: 0 0 1.8rem 0; }

    div[data-testid="stTextInput"] input {
        border-radius: 6px !important; padding: 0.7rem 0.9rem !important; font-size: 0.98rem
        !important;
        border: 1px solid var(--border) !important; box-shadow: none !important; transition:
        border-color .12s ease;
    }
    div[data-testid="stTextInput"] input:focus {
        border-color: var(--accent) !important; box-shadow: 0 0 3px rgba(29, 78, 216, 0.12)
        !important;
    }

    div[data-testid="stFormSubmitButton"] button {
        background: var(--accent-solid) !important; color: white !important; border: none
        !important;
        border-radius: 6px !important; padding: 0.6rem 1.4rem !important; font-weight: 600
        !important;
        letter-spacing: 0.01em; transition: background .12s ease;
    }
    div[data-testid="stFormSubmitButton"] button:hover { background: #1e40af !important; }
    div[data-testid="stFormSubmitButton"] button p { color: white !important; font-weight: 600;
    }

    div[class*="st-key-report_card"] {
        border-radius: 8px !important; border: 1px solid var(--border) !important;
        box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04); padding: 0.3rem 0.3rem;
    }
    """
)

```

```

div[class*"st-key-report_card"] h2 {
    font-size: 0.78rem !important; font-weight: 600 !important; text-transform: uppercase;
    letter-spacing: 0.08em;
    color: var(--muted) !important; border-bottom: 1px solid var(--border); padding-bottom:
    0.5rem; margin-bottom: 0.9rem !important;
}
div[class*"st-key-report_card"] h3 { font-size: 1rem !important; color: var(--ink)
!important; }

.query-label { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.08em; text-transform:
uppercase; color: var(--muted); margin-bottom: 0.15rem; }
.query-text { font-size: 1.05rem; font-weight: 600; color: var(--ink); margin-bottom: 1rem;
padding-bottom: 1rem; border-bottom: 1px solid var(--border); }

.sources-label { font-size: 0.78rem; font-weight: 600; text-transform: uppercase;
letter-spacing: 0.08em; color: var(--muted); margin: 1.2rem 0 0.7rem 0; }
div[class*"st-key-pg_"] { border-radius: 6px !important; border: 1px solid var(--border)
!important; background: var(--surface) !important; }
.src-row { display: flex; align-items: flex-start; gap: 0.65rem; }
.src-badge {
    flex: none; width: 1.5rem; height: 1.5rem; border-radius: 4px; background:
    var(--accent-solid); color: white;
    font-size: 0.72rem; font-weight: 600; display: flex; align-items: center;
    justify-content: center;
}
.src-title { font-weight: 600; font-size: 0.9rem; color: var(--ink); margin-bottom: 0.1rem;
}
.src-title a { text-decoration: none; color: inherit; }
.src-title a:hover { color: var(--accent); }
.src-domain { font-size: 0.72rem; color: var(--muted); text-transform: uppercase;
letter-spacing: 0.03em; }

.empty-state { padding: 2.4rem 1rem; text-align: center; color: var(--muted); border: 1px
dashed var(--border); border-radius: 8px; font-size: 0.92rem; }
</style>
"""
unsafe_allow_html=True,
)

st.markdown('<div class="eyebrow">Firecrawl · crawl()</div>', unsafe_allow_html=True)
st.markdown('<div class="hero-title">Exploration de Site</div>', unsafe_allow_html=True)
st.markdown(
    '<div class="hero-subtitle">Explore un site en suivant ses liens, lit le contenu de
    plusieurs pages '
    'et en produit une fiche de synthèse sourcée.</div>',
    unsafe_allow_html=True,
)
st.markdown('<hr class="hero-rule" />', unsafe_allow_html=True)

with st.sidebar:
    st.markdown("### Paramètres")
    limit = st.slider("Nombre de pages à explorer", min_value=2, max_value=10,
        value=CRAWL_LIMIT)
    st.caption("crawl() scrape réellement chaque page : gardez une limite basse pour économiser
        des crédits.")

with st.form("crawl_form"):
    url = st.text_input(
        "URL de départ :",
        placeholder="Ex : https://docs.firecrawl.dev/introduction",
        label_visibility="collapsed",
    )
    submitted = st.form_submit_button("Explorer le site")

if "result" not in st.session_state:
    st.session_state.result = None

if submitted and url.strip():
    with st.spinner("Exploration du site et lecture des pages en cours..."):
        report, pages = digest(url.strip(), limit=limit)
        st.session_state.result = {"url": url.strip(), "report": report, "pages": pages}

if not st.session_state.result:
    st.markdown(
        '<div class="empty-state">Aucune exploration pour l'instant. '
        'Saisissez une URL de départ ci-dessus pour commencer.</div>',
        unsafe_allow_html=True,
    )
else:
    res = st.session_state.result
    with st.container(border=True, key="report_card"):
        st.markdown('<div class="query-label">Site exploré</div>', unsafe_allow_html=True)
        st.markdown(f'<div class="query-text">{res["url"]}</div>', unsafe_allow_html=True)

```

```

st.markdown(res["report"])

if res["pages"]:
    st.markdown(f'<div class="sources-label">{len(res["pages"])} page(s)
    explorée(s)</div>', unsafe_allow_html=True)
    for i, p in enumerate(res["pages"]):
        domain = p.url.split("/")[2] if "/" in p.url else p.url
        with st.container(border=True, key=f"pg_{i}"):
            st.markdown(
                '<div class="src-row">'
                f'<div class="src-badge">{i + 1}</div>'
                '<div>'
                f'<div class="src-title"><a href="{p.url}"'
                target="_blank">{p.title}</a></div>'
                f'<div class="src-domain">{domain}</div>'
                '</div>'
                '</div>',
                unsafe_allow_html=True,
            )

```


40. PageAnalyzer-Firecrawl

40-PageAnalyzer-Firecrawl/scrape_agent.py

```

import os
from textwrap import dedent
from typing import Optional

from dotenv import load_dotenv
from firecrawl import Firecrawl

from agno.agent import Agent
from agno.models.groq import Groq

load_dotenv()

FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

# scrape() ne lit qu'UNE page : on peut se permettre de garder beaucoup plus de
# contenu que pour search()/crawl() (qui cumulent plusieurs pages dans le même
# prompt) tout en restant sous la limite Groq (8000 tokens/minute, tier gratuit).
MAX_CONTENT_CHARS = 12000

firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)

page_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Page Analyzer",
    role="Expert en analyse de contenu web",
    instructions=dedent("""
    Vous recevez le contenu markdown d'UNE page web, et soit une question précise,
    soit une demande de résumé général.

    - Si une question est posée : répondez-y en vous basant UNIQUEMENT sur le
    contenu fourni. Si la page ne contient pas la réponse, dites-le clairement
    plutôt que d'inventer.
    - Si aucune question n'est posée : produisez un résumé structuré en Markdown
    avec les points clés de la page.
    - Répondez dans la langue de la question (ou du contenu si pas de question).
    """),
    markdown=True,
)

def scrape_page(url: str) -> Optional[str]:
    """Scrape une page via Firecrawl et retourne son contenu markdown (None si échec)."""
    result = firecrawl.scrape(url, formats=["markdown"])
    markdown = getattr(result, "markdown", None)
    if not markdown or not markdown.strip():
        return None
    return markdown.strip()

def analyze_page(url: str, question: Optional[str] = None) -> tuple[Optional[str],
Optional[str]]:
    """Scrape `url` puis répond à `question` (ou résume la page si absente).

    Retourne (réponse, erreur) : l'un des deux est toujours None.
    """
    content = scrape_page(url)
    if content is None:
        return None, "Impossible d'extraire le contenu de cette page (site bloqué, JS-only, ou page vide)."

    truncated = content[:MAX_CONTENT_CHARS]
    if question and question.strip():
        prompt = f"Question : {question.strip()}\n\nContenu de la page ({url}) :\n\n{truncated}"
    else:
        prompt = f"Résume cette page ({url}) :\n\n{truncated}"

    answer = page_agent.run(prompt)
    return answer.content, None

```

40-PageAnalyzer-Firecrawl/app.py

```
import streamlit as st
from scrape_agent import analyze_page
st.set_page_config(page_title="Analyseur de Page", page_icon="◆", layout="wide")
```

```

st.markdown(
    """
    <style>
    @import
        url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

    :root {
        --ink: #0f172a; --muted: #64748b; --accent: #1d4ed8; --accent-solid: #1d4ed8;
        --border: #e2e8f0; --surface: rgba(15, 23, 42, 0.02); --card-bg: #ffffff;
    }
    @media (prefers-color-scheme: dark) {
        :root {
            --ink: #f1f5f9; --muted: #94a3b8; --accent: #60a5fa;
            --border: rgba(255, 255, 255, 0.14); --surface: rgba(255, 255, 255, 0.04);
            --card-bg: #1e293b;
        }
    }

    html, body, [class*="css"] { font-family: 'Inter', -apple-system, sans-serif; }
    h1, h2, h3 { font-family: 'Inter', sans-serif !important; font-weight: 700 !important; }
    .block-container { padding-top: 2.2rem; max-width: 880px; }

    .eyebrow { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.12em; text-transform:
        uppercase; color: var(--accent); margin-bottom: 0.5rem; }
    .hero-title { font-weight: 700; font-size: 2rem; color: var(--ink); margin-bottom: 0.35rem;
        line-height: 1.25; }
    .hero-subtitle { color: var(--muted); font-size: 0.98rem; margin-bottom: 1.4rem; max-width:
        640px; line-height: 1.5; }
    .hero-rule { border: none; border-top: 1px solid var(--border); margin: 0 0 1.8rem 0; }

    div[data-testid="stTextInput"] input {
        border-radius: 6px !important; padding: 0.7rem 0.9rem !important; font-size: 0.98rem
        !important;
        border: 1px solid var(--border) !important; box-shadow: none !important; transition:
            border-color .12s ease;
    }
    div[data-testid="stTextInput"] input:focus {
        border-color: var(--accent) !important; box-shadow: 0 0 3px rgba(29, 78, 216, 0.12)
        !important;
    }

    div[data-testid="stFormSubmitButton"] button {
        background: var(--accent-solid) !important; color: white !important; border: none
        !important;
        border-radius: 6px !important; padding: 0.6rem 1.4rem !important; font-weight: 600
        !important;
        letter-spacing: 0.01em; transition: background .12s ease;
    }
    div[data-testid="stFormSubmitButton"] button:hover { background: #1e40af !important; }
    div[data-testid="stFormSubmitButton"] button p { color: white !important; font-weight: 600;
    }

    div[class*="st-key-report_card"] {
        border-radius: 8px !important; border: 1px solid var(--border) !important;
        box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04); padding: 0.3rem 0.3rem;
    }
    div[class*="st-key-report_card"] h2 {
        font-size: 0.78rem !important; font-weight: 600 !important; text-transform: uppercase;
        letter-spacing: 0.08em;
        color: var(--muted) !important; border-bottom: 1px solid var(--border); padding-bottom:
            0.5rem; margin-bottom: 0.9rem !important;
    }
    div[class*="st-key-report_card"] h3 { font-size: 1rem !important; color: var(--ink)
        !important; }

    .query-label { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.08em; text-transform:
        uppercase; color: var(--muted); margin-bottom: 0.15rem; }
    .query-text { font-size: 1.05rem; font-weight: 600; color: var(--ink); margin-bottom: 1rem;
        padding-bottom: 1rem; border-bottom: 1px solid var(--border); }

    .empty-state { padding: 2.4rem 1rem; text-align: center; color: var(--muted); border: 1px
        dashed var(--border); border-radius: 8px; font-size: 0.92rem; }
    .error-box { padding: 1rem 1.1rem; border: 1px solid rgba(220, 38, 38, 0.3); background:
        rgba(220, 38, 38, 0.06); color: #dc2626; border-radius: 8px; font-size: 0.9rem; }
    </style>
    """
    unsafe_allow_html=True,
)

st.markdown('<div class="eyebrow">Firecrawl · scrape()</div>', unsafe_allow_html=True)
st.markdown('<div class="hero-title">Analyseur de Page</div>', unsafe_allow_html=True)
st.markdown(
    '<div class="hero-subtitle">Lit le contenu d’une page web et répond à une question précise à

```

```

        son sujet, '
        'ou en produit un résumé si aucune question n'est posée.</div>';
        unsafe_allow_html=True,
    )
    st.markdown('<hr class="hero-rule" />', unsafe_allow_html=True)

    with st.form("page_form"):
        url = st.text_input("URL de la page :", placeholder="Ex :
            https://docs.firecrawl.dev/introduction")
        question = st.text_input("Question (optionnel) :", placeholder="Laisser vide pour un résumé
            général")
        submitted = st.form_submit_button("Analyser la page")

    if "result" not in st.session_state:
        st.session_state.result = None

    if submitted and url.strip():
        with st.spinner("Lecture de la page en cours..."):
            answer, error = analyze_page(url.strip(), question=question.strip() or None)
            st.session_state.result = {"url": url.strip(), "question": question.strip(), "answer":
                answer, "error": error}

    if not st.session_state.result:
        st.markdown(
            '<div class="empty-state">Aucune analyse pour l'instant. '
            "Saisissez une URL ci-dessus pour commencer.</div>",
            unsafe_allow_html=True,
        )
    else:
        res = st.session_state.result
        if res["error"]:
            st.markdown(f'<div class="error-box">{res["error"]}</div>', unsafe_allow_html=True)
        else:
            with st.container(border=True, key="report_card"):
                st.markdown('<div class="query-label">Page analysée</div>', unsafe_allow_html=True)
                label = res["question"] if res["question"] else "Résumé général"
                st.markdown(
                    f'<div class="query-text">{res["url"]}<br>'
                    f'<span style="font-weight:400; font-size:0.85rem;'
                    color:var(--muted);">{label}</span></div>',
                    unsafe_allow_html=True,
                )
                st.markdown(res["answer"])

```

41. DocumentCompare-MistralOCR

41-DocumentCompare-MistralOCR/compare_agent.py

```
import os
from textwrap import dedent
from typing import Literal

from dotenv import load_dotenv
from mistralai.client import Mistral
from pydantic import BaseModel, Field

from agno.agent import Agent
from agno.models.mistral import MistralChat

load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")

# mistral-large-latest a un contexte large, mais on garde une limite raisonnable
# par document pour que la comparaison reste rapide et abordable même sur de
# gros contrats.
MAX_CHARS_PER_DOC = 15000

mistral_client = Mistral(api_key=MISTRAL_API_KEY)

class Change(BaseModel):
    type: Literal["Ajout", "Suppression", "Modification"] = Field(
        description="Nature du changement entre le document A et le document B"
    )
    clause: str = Field(description="Section ou clause concernée (titre court)")
    detail: str = Field(description="Explication précise de ce qui a changé, en citant le
        contenu avant/après si pertinent")

class ComparisonResult(BaseModel):
    summary: str = Field(description="Résumé en 2-3 phrases de l'ampleur et de la nature globale
        des changements")
    changes: list[Change] = Field(description="Liste de tous les changements identifiés entre
        les deux documents")

comparison_agent = Agent(
    model=MistralChat(api_key=MISTRAL_API_KEY, id="mistral-large-latest"),
    name="Document Comparator",
    role="Expert en analyse comparative de documents (contrats, textes légaux, versions
        successives)",
    instructions=dedent("""
    Vous recevez le contenu de deux versions d'un même document (Document A = version
    précédente, Document B = version actuelle), tous deux extraits par OCR.

    Votre mission : identifier PRÉCISÉMENT ce qui a changé entre les deux versions.

    Règles strictes :
    - Ne signalez que des changements de FOND (clauses, montants, dates, obligations,
        conditions...). Ignorez les différences purement cosmétiques (mise en page,
        espacement, artefacts d'OCR sans impact sur le sens).
    - Pour chaque changement, précisez son type : "Ajout" (nouveau dans B, absent de A),
        "Suppression" (présent dans A, absent de B), ou "Modification" (présent dans les
        deux mais différent).
    - Soyez précis et factuel : citez ce qui a changé, pas une impression vague.
    - Si les deux documents sont identiques sur le fond, renvoyez une liste de
        changements vide et dites-le clairement dans le résumé.
    - Ne signalez jamais un changement que vous ne pouvez pas justifier avec le texte
        fourni.
    """),
    output_schema=ComparisonResult,
)

def ocr_pdf(file_path: str) -> str:
    """Extrait le contenu markdown d'un PDF via l'OCR Mistral."""
    uploaded = mistral_client.files.upload(
        file={"file_name": os.path.basename(file_path), "content": open(file_path, "rb")},
        purpose="ocr",
    )
    signed_url = mistral_client.files.get_signed_url(file_id=uploaded.id)
    ocr_response = mistral_client.ocr.process(
        model="mistral-ocr-latest",
        document={"type": "document_url", "document_url": signed_url.url},
    )
```

```

        include_image_base64=False,
    )
    return "\n".join(page.markdown for page in ocr_response.pages)

def compare_documents(text_a: str, text_b: str) -> ComparisonResult:
    """Compare deux textes de documents et retourne les changements structurés."""
    prompt = (
        f"Document A (version précédente) :\n\n{text_a[:MAX_CHARS_PER_DOC]}\n\n"
        f"---\n\n"
        f"Document B (version actuelle) :\n\n{text_b[:MAX_CHARS_PER_DOC]}"
    )
    result = comparison_agent.run(prompt)
    if not isinstance(result.content, ComparisonResult):
        return ComparisonResult(
            summary="La comparaison n'a pas pu être structurée correctement, réessayez.",
            changes=[],
        )
    return result.content

def compare_pdfs(path_a: str, path_b: str) -> tuple[ComparisonResult, str, str]:
    """OCR les deux PDFs puis les compare. Retourne (résultat, texte_a, texte_b)."""
    text_a = ocr_pdf(path_a)
    text_b = ocr_pdf(path_b)
    result = compare_documents(text_a, text_b)
    return result, text_a, text_b

```

41-DocumentCompare-MistralOCR/app.py

```

import os

import streamlit as st
from compare_agent import compare_documents, ocr_pdf

st.set_page_config(page_title="Comparateur de Documents", page_icon="♦", layout="wide")

st.markdown(
    """
    <style>
    @import
        url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

    :root {
        --ink: #0f172a; --muted: #64748b; --accent: #1d4ed8; --accent-solid: #1d4ed8;
        --border: #e2e8f0; --surface: rgba(15, 23, 42, 0.02); --card-bg: #ffffff;
        --green: #16a34a; --green-bg: rgba(22, 163, 74, 0.1);
        --red: #dc2626; --red-bg: rgba(220, 38, 38, 0.1);
        --amber: #d97706; --amber-bg: rgba(217, 119, 6, 0.1);
    }
    @media (prefers-color-scheme: dark) {
        :root {
            --ink: #f1f5f9; --muted: #94a3b8; --accent: #60a5fa;
            --border: rgba(255, 255, 255, 0.14); --surface: rgba(255, 255, 255, 0.04);
            --card-bg: #1e293b;
            --green: #4ade80; --green-bg: rgba(74, 222, 128, 0.12);
            --red: #f87171; --red-bg: rgba(248, 113, 113, 0.12);
            --amber: #fbbf24; --amber-bg: rgba(251, 191, 36, 0.12);
        }
    }

    html, body, [class*="css"] { font-family: 'Inter', -apple-system, sans-serif; }
    h1, h2, h3 { font-family: 'Inter', sans-serif !important; font-weight: 700 !important; }
    .block-container { padding-top: 2.2rem; max-width: 900px; }

    .eyebrow { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.12em; text-transform:
        uppercase; color: var(--accent); margin-bottom: 0.5rem; }
    .hero-title { font-weight: 700; font-size: 2rem; color: var(--ink); margin-bottom: 0.35rem;
        line-height: 1.25; }
    .hero-subtitle { color: var(--muted); font-size: 0.98rem; margin-bottom: 1.4rem; max-width:
        660px; line-height: 1.5; }
    .hero-rule { border: none; border-top: 1px solid var(--border); margin: 0 0 1.8rem 0; }

    div[data-testid="stFormSubmitButton"] button {
        background: var(--accent-solid) !important; color: white !important; border: none
        !important;
        border-radius: 6px !important; padding: 0.6rem 1.4rem !important; font-weight: 600
        !important;
        letter-spacing: 0.01em; transition: background .12s ease;
    }
    div[data-testid="stFormSubmitButton"] button:hover { background: #1e40af !important; }
    div[data-testid="stFormSubmitButton"] button p { color: white !important; font-weight: 600;

```

```

    }

.doc-label { font-size: 0.78rem; font-weight: 600; text-transform: uppercase;
    letter-spacing: 0.06em; color: var(--muted); margin-bottom: 0.4rem; }

div[class*="st-key-summary_card"] {
    border-radius: 8px !important; border: 1px solid var(--border) !important;
    box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04); background: var(--surface) !important;
}
.summary-label { font-size: 0.72rem; font-weight: 600; text-transform: uppercase;
    letter-spacing: 0.08em; color: var(--muted); margin-bottom: 0.5rem; }
.summary-text { color: var(--ink); font-size: 0.98rem; line-height: 1.5; }

div[class*="st-key-chg_"] {
    border-radius: 8px !important; border: 1px solid var(--border) !important;
    box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04);
}
.chg-badge {
    display: inline-block; font-size: 0.68rem; font-weight: 700; text-transform: uppercase;
    letter-spacing: 0.04em; border-radius: 4px; padding: 0.15rem 0.5rem; margin-bottom:
        0.5rem;
}
.chg-badge.Ajout { color: var(--green); background: var(--green-bg); }
.chg-badge.Suppression { color: var(--red); background: var(--red-bg); }
.chg-badge.Modification { color: var(--amber); background: var(--amber-bg); }
.chg-clause { font-weight: 600; font-size: 0.95rem; color: var(--ink); margin-bottom:
    0.3rem; }
.chg-detail { font-size: 0.87rem; color: var(--muted); line-height: 1.5; }

.empty-state { padding: 2.4rem 1rem; text-align: center; color: var(--muted); border: 1px
    dashed var(--border); border-radius: 8px; font-size: 0.92rem; }
.no-changes { padding: 1.2rem; text-align: center; color: var(--green); background:
    var(--green-bg); border-radius: 8px; font-size: 0.92rem; font-weight: 500; }
.error-box { padding: 1rem 1.1rem; border: 1px solid rgba(220, 38, 38, 0.3); background:
    var(--red-bg); color: var(--red); border-radius: 8px; font-size: 0.9rem; }
</style>
"""
unsafe_allow_html=True,
)

st.markdown('<div class="eyebrow">Mistral OCR · Comparaison</div>', unsafe_allow_html=True)
st.markdown('<div class="hero-title">Comparateur de Documents</div>', unsafe_allow_html=True)
st.markdown(
    '<div class="hero-subtitle">Extrait le texte de deux versions d'un document par OCR '
    'et identifie précisément les clauses ajoutées, supprimées ou modifiées entre les '
    'deux.</div>',
    unsafe_allow_html=True,
)
st.markdown('<hr class="hero-rule" />', unsafe_allow_html=True)

with st.form("compare_form"):
    col_a, col_b = st.columns(2)
    with col_a:
        st.markdown('<div class="doc-label">Document A – version précédente</div>',
            unsafe_allow_html=True)
        file_a = st.file_uploader("Document A", type=["pdf"], label_visibility="collapsed",
            key="file_a")
    with col_b:
        st.markdown('<div class="doc-label">Document B – version actuelle</div>',
            unsafe_allow_html=True)
        file_b = st.file_uploader("Document B", type=["pdf"], label_visibility="collapsed",
            key="file_b")
    submitted = st.form_submit_button("Comparer les documents")

if "result" not in st.session_state:
    st.session_state.result = None

if submitted:
    if not file_a or not file_b:
        st.markdown('<div class="error-box">Merci d'ajouter les deux documents avant de lancer '
            'la comparaison.</div>', unsafe_allow_html=True)
    else:
        os.makedirs("uploads", exist_ok=True)
        path_a = os.path.join("uploads", file_a.name)
        path_b = os.path.join("uploads", file_b.name)
        with open(path_a, "wb") as f:
            f.write(file_a.read())
        with open(path_b, "wb") as f:
            f.write(file_b.read())

        try:
            with st.spinner("Extraction du texte (OCR) des deux documents..."):
                text_a = ocr_pdf(path_a)

```

```

        text_b = ocr_pdf(path_b)
        with st.spinner("Comparaison des deux versions en cours..."):
            result = compare_documents(text_a, text_b)
            st.session_state.result = {"name_a": file_a.name, "name_b": file_b.name, "result":
            result}
        except Exception as e:
            st.session_state.result = {"error": str(e)}

if not st.session_state.result:
    st.markdown(
        '<div class="empty-state">Aucune comparaison pour l'instant. '
        'Ajoutez les deux versions d'un document ci-dessus pour commencer.</div>',
        unsafe_allow_html=True,
    )
elif st.session_state.result.get("error"):
    st.markdown(f'<div class="error-box">Erreur pendant la comparaison :
    {st.session_state.result["error"]}</div>', unsafe_allow_html=True)
else:
    res = st.session_state.result
    comparison = res["result"]

    with st.container(border=True, key="summary_card"):
        st.markdown('<div class="summary-label">Résumé</div>', unsafe_allow_html=True)
        st.markdown(f'<div class="summary-text">{comparison.summary}</div>',
            unsafe_allow_html=True)
        st.caption(f'{res["name_a"]} → {res["name_b"]}')

    st.write("")

    if not comparison.changes:
        st.markdown('<div class="no-changes">Aucun changement de fond détecté entre les deux
        versions.</div>', unsafe_allow_html=True)
    else:
        st.markdown(f'***{len(comparison.changes)} changement(s) détecté(s)***')
        for i, change in enumerate(comparison.changes):
            with st.container(border=True, key=f"chg_{i}"):
                st.markdown(
                    f'<span class="chg-badge {change.type}>{change.type}</span>'
                    f'<div class="chg-clause">{change.clause}</div>'
                    f'<div class="chg-detail">{change.detail}</div>',
                    unsafe_allow_html=True,
                )

```

42. ResearchTeam-Agno

42-ResearchTeam-Agno/research_team.py

```
import os
from textwrap import dedent
from typing import Optional

from dotenv import load_dotenv
from firecrawl import Firecrawl
from firecrawl.v2.types import ScrapeOptions

from agno.agent import Agent
from agno.team.team import Team
from agno.models.groq import Groq

load_dotenv()

FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
# Le 120B est plus capable mais son quota Groq gratuit (8000 tokens/minute) est
# vite dépassé dès qu'on cumule recherche + vérification + rédaction dans une
# même minute ; le 20B a un quota nettement plus élevé (voir 20-Agno-Team pour
# le même constat). Tous les membres l'utilisent pour rester fiables ensemble.
MODEL_ID = "openai/gpt-oss-20b"

def groq_model(model_id: str = MODEL_ID) -> Groq:
    """Groq configuré avec un léger retry : un test réel avec 3 membres + un
    Vérificateur séparé a déclenché 96 dépassements de quota Groq (8000
    tokens/minute, tier gratuit) sur une seule exécution – chaque agent relaie
    tout le contenu précédent au suivant, donc le coût cumulé explose vite. Un
    retry court aide sur les dépassements ponctuels, mais la vraie solution a
    été de réduire l'équipe à 2 membres (voir plus bas).
    """
    return Groq(
        id=model_id,
        api_key=GROQ_API_KEY,
        retries=2,
        delay_between_retries=2,
        exponential_backoff=True,
    )

# Équipe à 2 membres (pas 3) et contenu volontairement compact : sur le tier
# gratuit Groq, chaque agent supplémentaire dans la chaîne "paie" à nouveau
# tout le contenu des étapes précédentes, ce qui fait exploser le total de
# tokens/minute bien plus vite qu'on ne l'imaginerait en ne regardant que la
# taille des sources elles-mêmes.
SEARCH_LIMIT = 3
MAX_CHARS_PER_SOURCE = 900

firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)

def firecrawl_search(query: str) -> str:
    """Recherche sur le web via Firecrawl et renvoie le contenu (markdown) des
    meilleurs résultats, numérotés [1], [2]... pour permettre des citations.
    Utilisez cet outil pour trouver de l'information factuelle et à jour sur
    n'importe quel sujet. Vous pouvez l'appeler plusieurs fois avec des requêtes
    différentes si le sujet a plusieurs facettes.
    """
    result = firecrawl.search(query, limit=SEARCH_LIMIT,
                              scrape_options=ScrapeOptions(formats=["markdown"]))

    blocks = []
    for i, item in enumerate(result.web or [], 1):
        markdown = (getattr(item, "markdown", None) or "").strip()
        metadata = getattr(item, "metadata", None)
        url = getattr(metadata, "url", None) if metadata else None
        title = (getattr(metadata, "title", None) if metadata else None) or url
        if not url or not markdown:
            continue
        blocks.append(f"[{i}] {title} ({url})\n{markdown[:MAX_CHARS_PER_SOURCE]}")

    return "\n\n".join(blocks) if blocks else "Aucun résultat exploitable trouvé pour cette
    recherche."

# --- Agent 1 : Chercheur -----
# Seul membre à avoir un outil : contrairement à l'agent 37 (recherche
# précalculée en Python avant l'appel LLM), c'est ici le LLM lui-même qui
```



```

# décide quand et combien de fois chercher.
searcher_agent = Agent(
    model=groq_model(),
    name="Chercheur",
    role="Spécialiste de la recherche web",
    instructions=dedent("""
Vous êtes chargé de rassembler de l'information factuelle et à jour sur le
sujet demandé. Utilisez l'outil `firecrawl_search` au moins une fois – plusieurs
fois si le sujet a plusieurs facettes ou angles distincts.

Présentez votre résultat comme une liste de sources numérotées [1], [2]...
avec, pour chacune : titre, URL, et les faits pertinents qu'elle contient.

Ne résumez pas et ne tirez aucune conclusion : votre rôle est de RASSEMBLER
la matière première brute, pas de la rédiger. Conservez la numérotation et
les URLs exactes fournies par l'outil.
"""),
    tools=[firecrawl_search],
    # Les résultats bruts de firecrawl_search (plusieurs pages scrapées) peuvent
    # être volumineux : on les compresse pour rester sous la limite Groq
    # (8000 tokens/minute, tier gratuit).
    compress_tool_results=True,
    markdown=True,
)

# --- Agent 2 : Rédacteur / Vérificateur -----
# La vérification est fusionnée dans les instructions du Rédacteur plutôt que
# confiée à un 3e agent séparé : un membre de plus dans la chaîne signifiait un
# aller-retour complet de plus via le leader, avec tout le contenu déjà
# accumulé à repayer en tokens – c'est ce qui faisait exploser le quota Groq.
writer_agent = Agent(
    model=groq_model(),
    name="Rédacteur",
    role="Rédacteur de rapports de synthèse sourcés, avec vérification croisée des faits",
    instructions=dedent("""
Vous recevez les sources numérotées rassemblées par le Chercheur. Rédigez le
rapport final destiné à l'utilisateur, en Markdown, en deux temps :

1. Vérifiez d'abord mentalement si les sources se corroborent ou se
   contredisent entre elles, et si certaines affirmations ne reposent que
   sur une seule source faible.
2. Rédigez ensuite le rapport avec cette structure :

## Synthèse
<lt;réponse structurée au sujet demandé, avec citations [1] [2]...>

## Points à nuancer
<lt;affirmations contradictoires entre sources, ou faiblement soutenues –
omettez cette section s'il n'y en a aucune>

## Sources
[1] <lt;titre> – <lt;url>
[2] <lt;titre> – <lt;url>
...

N'inventez rien qui ne soit pas dans les sources fournies par le Chercheur.
"""),
    markdown=True,
)

research_team = Team(
    members=[searcher_agent, writer_agent],
    # Le leader orchestre 2 échanges successifs (délégation à chaque membre) : on
    # utilise le modèle 20B, au quota Groq plus élevé que le 120B, pour éviter de
    # dépasser la limite de 8000 tokens/minute du tier gratuit sur ce cumul.
    model=groq_model(),
    mode="coordinate",
    # Force la délégation aux 2 membres dans l'ordre : le rapport final a besoin
    # des 2 apports, on ne laisse pas le leader décider de sauter une étape.
    delegate_to_all_members=True,
    instructions=dedent("""
Vous coordonnez une équipe de 2 agents pour produire un rapport de recherche
web fiable et sourcé sur le sujet demandé par l'utilisateur.

Ordre de travail impératif :
1. Déléguez d'abord au Chercheur : il doit rassembler des sources via son
   outil de recherche web.
2. Déléguez ensuite au Rédacteur, en lui transmettant EXACTEMENT les
   sources rassemblées par le Chercheur (numérotation et URLs incluses) :
   il vérifie leur cohérence et rédige le rapport final.

Renvoyez en sortie finale UNIQUEMENT le rapport produit par le Rédacteur
(ne le résumez pas, ne le raccourcissez pas, ne le reformulez pas).
""")

```

```

    """),
    markdown=True,
)

def run_research(query: str) -> tuple[Optional[str], Optional[str]]:
    """Lance l'équipe sur `query`. Retourne (rapport, erreur) : l'un des deux est
    toujours None.

    Nécessaire car un échec d'appel Groq (ex: quota TPM dépassé) ne lève pas
    toujours d'exception ici : Agno peut renvoyer le message d'erreur brut de
    l'API (JSON commençant par '{"error":}') directement comme `result.content`,
    ce qui l'afficherait tel quel dans l'UI sans ce garde-fou.
    """
    try:
        result = research_team.run(query)
    except Exception as e:
        return None, f"Erreur pendant l'exécution de l'équipe : {e}"

    content = result.content
    if not isinstance(content, str) or not content.strip():
        return None, "L'équipe n'a pas produit de rapport exploitable, réessayez."
    if content.strip().startswith('{"error":}'):
        return None, "Le fournisseur du modèle a temporairement refusé la requête (quota
        atteint). Réessayez dans quelques instants."
    return content, None

```

42 - ResearchTeam-Agno/app.py

```

import streamlit as st
from research_team import run_research

st.set_page_config(page_title="Équipe de Recherche", page_icon="◆", layout="wide")

st.markdown(
    """
    <style>
    @import
        url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

    :root {
        --ink: #0f172a; --muted: #64748b; --accent: #1d4ed8; --accent-solid: #1d4ed8;
        --border: #e2e8f0; --surface: rgba(15, 23, 42, 0.02); --card-bg: #ffffff;
    }
    @media (prefers-color-scheme: dark) {
        :root {
            --ink: #f1f5f9; --muted: #94a3b8; --accent: #60a5fa;
            --border: rgba(255, 255, 255, 0.14); --surface: rgba(255, 255, 255, 0.04);
            --card-bg: #1e293b;
        }
    }

    html, body, [class*="css"] { font-family: 'Inter', -apple-system, sans-serif; }
    h1, h2, h3 { font-family: 'Inter', sans-serif !important; font-weight: 700 !important; }
    .block-container { padding-top: 2.2rem; max-width: 880px; }

    .eyebrow { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.12em; text-transform:
        uppercase; color: var(--accent); margin-bottom: 0.5rem; }
    .hero-title { font-weight: 700; font-size: 2rem; color: var(--ink); margin-bottom: 0.35rem;
        line-height: 1.25; }
    .hero-subtitle { color: var(--muted); font-size: 0.98rem; margin-bottom: 1.2rem; max-width:
        660px; line-height: 1.5; }
    .hero-rule { border: none; border-top: 1px solid var(--border); margin: 0 0 1.8rem 0; }

    .team-row { display: flex; gap: 0.6rem; margin-bottom: 1.6rem; flex-wrap: wrap; }
    .team-chip {
        display: flex; align-items: center; gap: 0.4rem; font-size: 0.82rem; color:
            var(--muted);
        border: 1px solid var(--border); border-radius: 999px; padding: 0.35rem 0.8rem;
        background: var(--surface);
    }
    .team-chip b { color: var(--ink); font-weight: 600; }
    .team-arrow { color: var(--border); align-self: center; }

    div[data-testid="stTextInput"] input {
        border-radius: 6px !important; padding: 0.7rem 0.9rem !important; font-size: 0.98rem
        !important;
        border: 1px solid var(--border) !important; box-shadow: none !important; transition:
            border-color .12s ease;
    }
    div[data-testid="stTextInput"] input:focus {
        border-color: var(--accent) !important; box-shadow: 0 0 0 3px rgba(29, 78, 216, 0.12)
    }

```

```

        !important;
    }

    div[data-testid="stFormSubmitButton"] button {
        background: var(--accent-solid) !important; color: white !important; border: none
        !important;
        border-radius: 6px !important; padding: 0.6rem 1.4rem !important; font-weight: 600
        !important;
        letter-spacing: 0.01em; transition: background .12s ease;
    }
    div[data-testid="stFormSubmitButton"] button:hover { background: #1e40af !important; }
    div[data-testid="stFormSubmitButton"] button p { color: white !important; font-weight: 600;
    }
}

div[class*="st-key-report_card"] {
    border-radius: 8px !important; border: 1px solid var(--border) !important;
    box-shadow: 0 1px 2px rgba(15, 23, 42, 0.04); padding: 0.3rem 0.3rem;
}
div[class*="st-key-report_card"] h2 {
    font-size: 0.78rem !important; font-weight: 600 !important; text-transform: uppercase;
    letter-spacing: 0.08em;
    color: var(--muted) !important; border-bottom: 1px solid var(--border); padding-bottom:
    0.5rem; margin-bottom: 0.9rem !important;
}
div[class*="st-key-report_card"] h3 { font-size: 1rem !important; color: var(--ink)
!important; }

.query-label { font-size: 0.72rem; font-weight: 600; letter-spacing: 0.08em; text-transform:
uppercase; color: var(--muted); margin-bottom: 0.15rem; }
.query-text { font-size: 1.05rem; font-weight: 600; color: var(--ink); margin-bottom: 1rem;
padding-bottom: 1rem; border-bottom: 1px solid var(--border); }

.empty-state { padding: 2.4rem 1rem; text-align: center; color: var(--muted); border: 1px
dashed var(--border); border-radius: 8px; font-size: 0.92rem; }
.error-box { padding: 1rem 1.1rem; border: 1px solid rgba(220, 38, 38, 0.3); background:
rgba(220, 38, 38, 0.06); color: #dc2626; border-radius: 8px; font-size: 0.9rem; }
</style>
""",
unsafe_allow_html=True,
)

st.markdown('<div class="eyebrow">Agno Team · 2 agents coordonnés</div>',
unsafe_allow_html=True)
st.markdown('<div class="hero-title">Équipe de Recherche</div>', unsafe_allow_html=True)
st.markdown(
    '<div class="hero-subtitle">Deux agents collaborent pour produire un rapport fiable : '
    'un Chercheur rassemble les sources via Firecrawl, un Rédacteur recoupe les faits et rédige
    le rapport final.</div>',
    unsafe_allow_html=True,
)
st.markdown(
    '<div class="team-row">'
    '<div class="team-chip">■ <b>Chercheur</b> – cherche && scrape via Firecrawl</div>'
    '<div class="team-arrow">→</div>'
    '<div class="team-chip">\\ <b>Rédacteur</b> – vérifie && rédige le rapport final</div>'
    '</div>',
    unsafe_allow_html=True,
)
st.markdown('<hr class="hero-rule" />', unsafe_allow_html=True)

with st.form("team_form"):
    query = st.text_input(
        "Sujet de recherche :",
        placeholder="Ex : Quelles sont les tendances 2026 du marché des VE en Europe ?",
        label_visibility="collapsed",
    )
    submitted = st.form_submit_button("Lancer l'équipe")

if "result" not in st.session_state:
    st.session_state.result = None

if submitted and query.strip():
    with st.spinner("L'équipe travaille : recherche → vérification → rédaction (peut prendre 1-2
    min)..."):
        report, error = run_research(query.strip())
        st.session_state.result = {"query": query.strip(), "report": report, "error": error}

if not st.session_state.result:
    st.markdown(
        '<div class="empty-state">Aucune recherche pour l'instant. '
        'Saisissez un sujet ci-dessus pour lancer l'équipe.</div>',
        unsafe_allow_html=True,
    )

```

```
elif st.session_state.result["error"]:
    st.markdown(f'&lt;div class="error-box"&gt;Erreur pendant l'exécution de l'équipe :
                {st.session_state.result["error"]}&lt;/div&gt;', unsafe_allow_html=True)
else:
    res = st.session_state.result
    with st.container(border=True, key="report_card"):
        st.markdown('&lt;div class="query-label"&gt;Sujet&lt;/div&gt;', unsafe_allow_html=True)
        st.markdown(f'&lt;div class="query-text"&gt;{res["query"]}&lt;/div&gt;', unsafe_allow_html=True)
        st.markdown(res["report"])
```